CST-407 Activity 7 Cross-Site Script Injection

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/1/2025

Table of Contents

Step 1 — Run the Victim “Comments” App (Port 8080)

1.1 Import & build

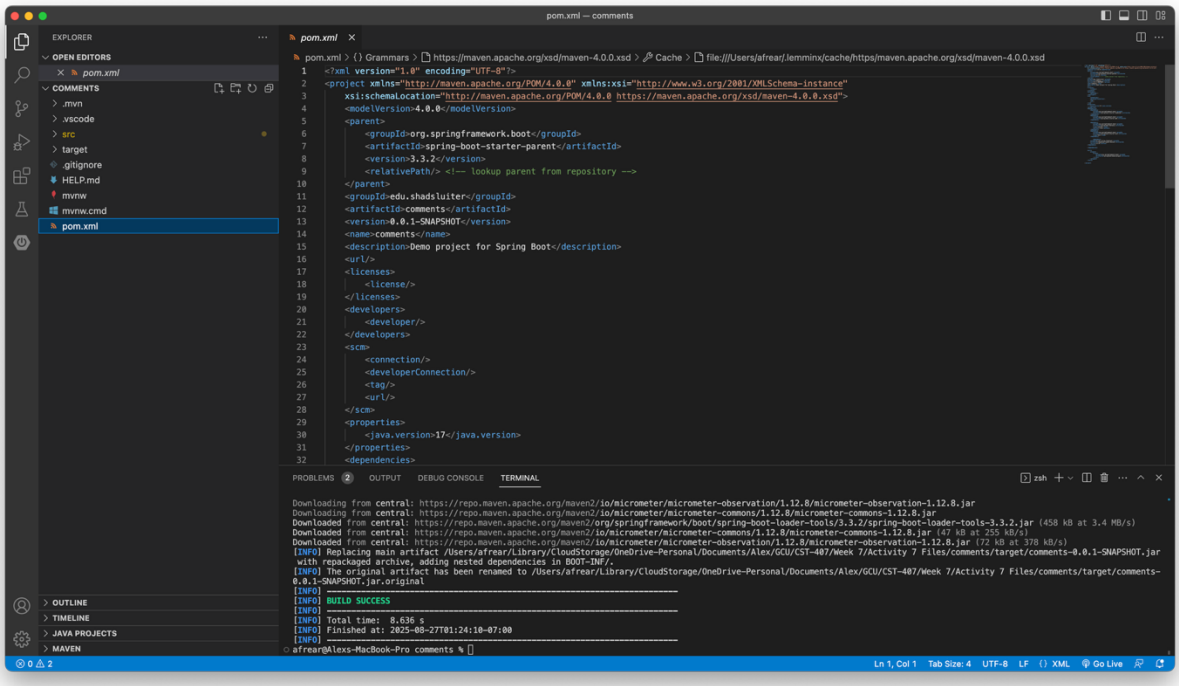

Figure 1 Project structure loaded with `pom.xml`, `src/main/java`, and `src/main/resources/templates/`.

1.2 Launch on 8080

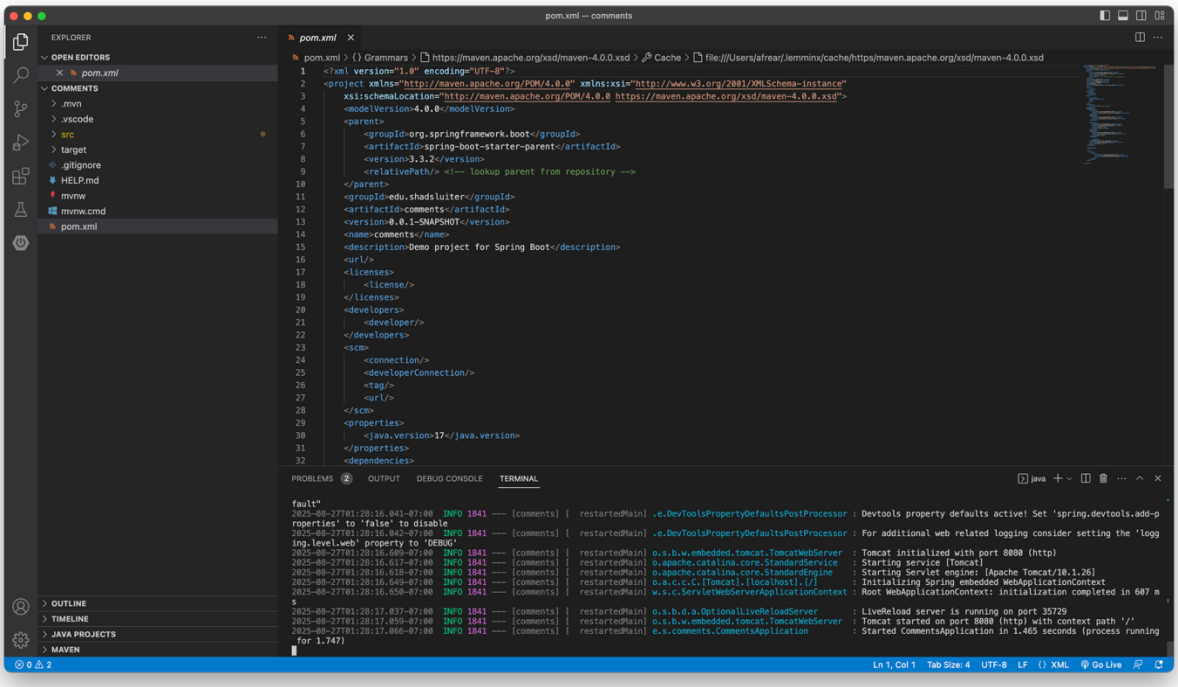

Figure 2 Spring Boot started; Tomcat listening on port 8080.

1.3 Open the UI

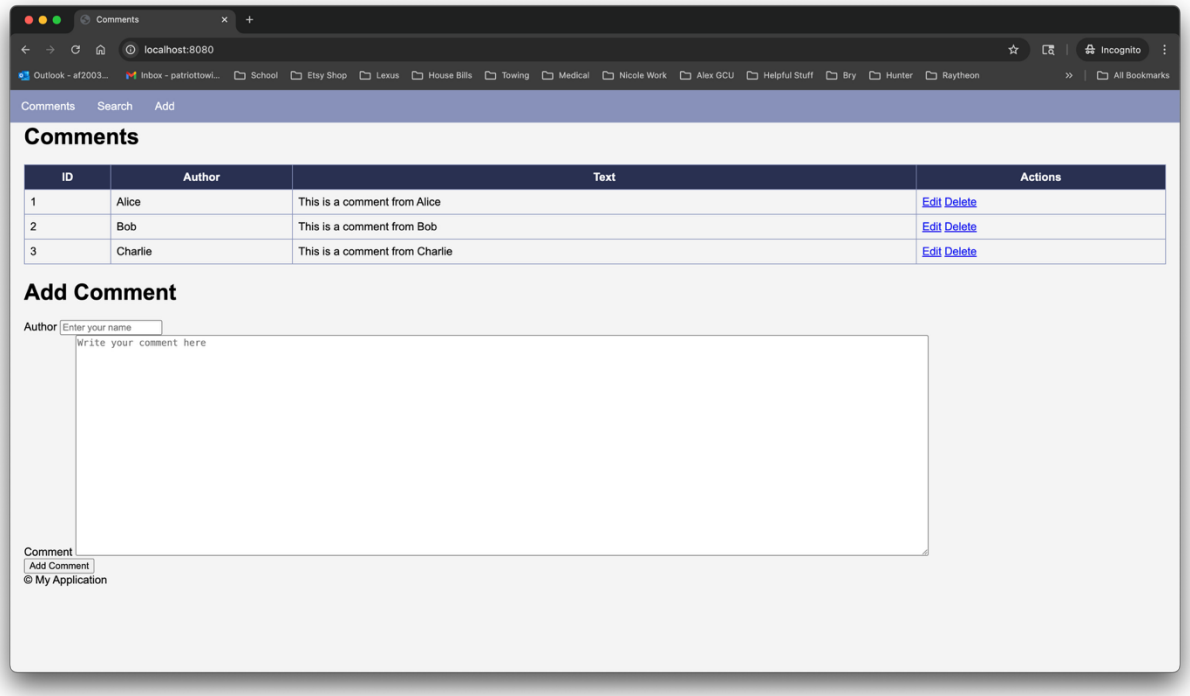

Figure 3 Comments page loaded with empty/initial table and Add Comment form.

Step 2 — Build & Run the Listener (“hacker”) app on **8081**

2.1 Generate a fresh Spring Boot project

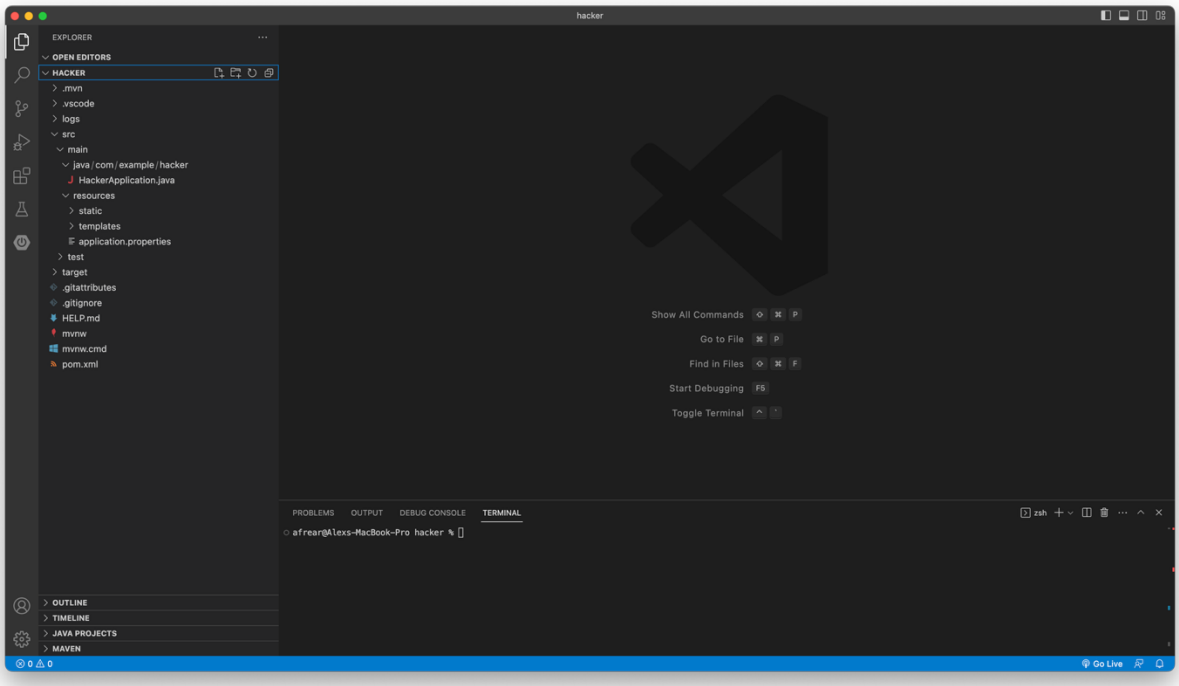

Figure 4 Project structure for the hacker listener app with controller and config files.

2.2 Run the app

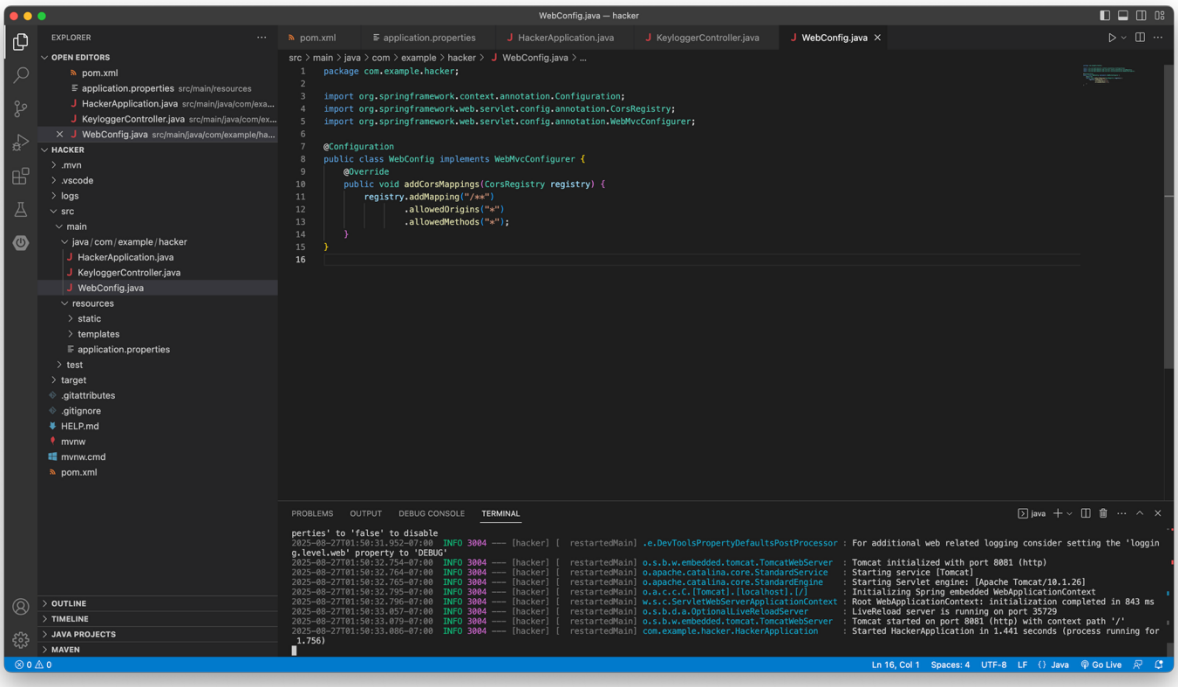

Figure 5 Spring Boot started successfully; Tomcat listening on port 8081.

2.3 Test the endpoints

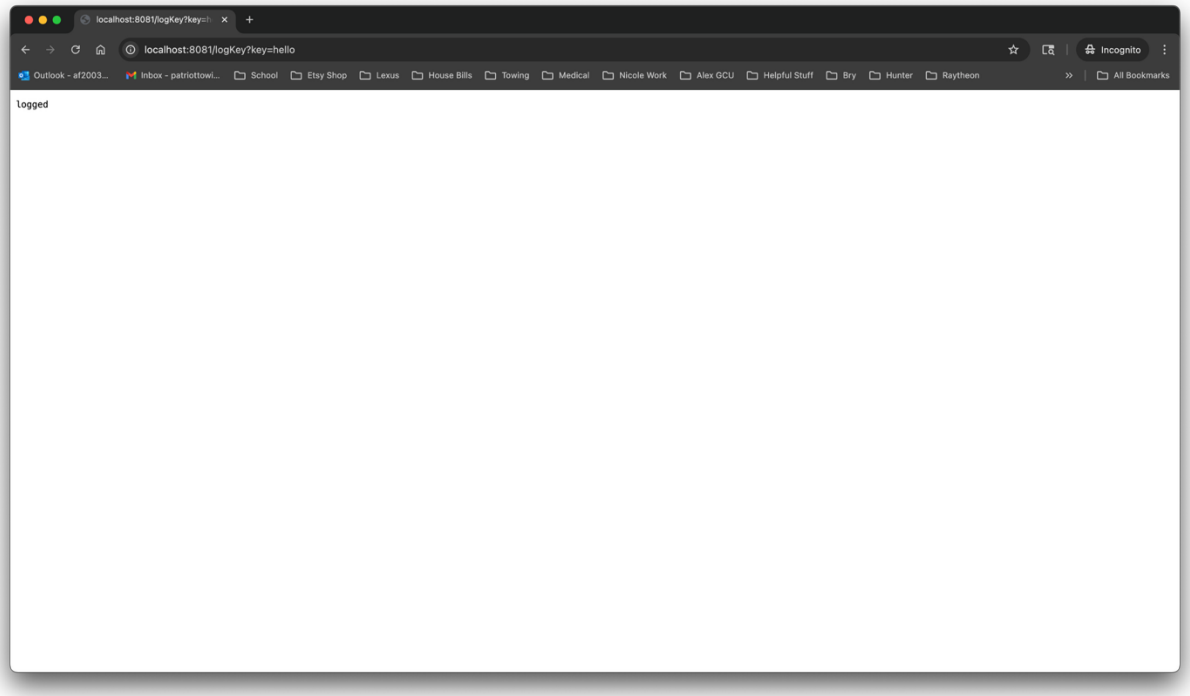

Figure 6 Browser request to `/logKey?key=hello` returns "logged."

2.4 Verify logs

Listener Console Evidence

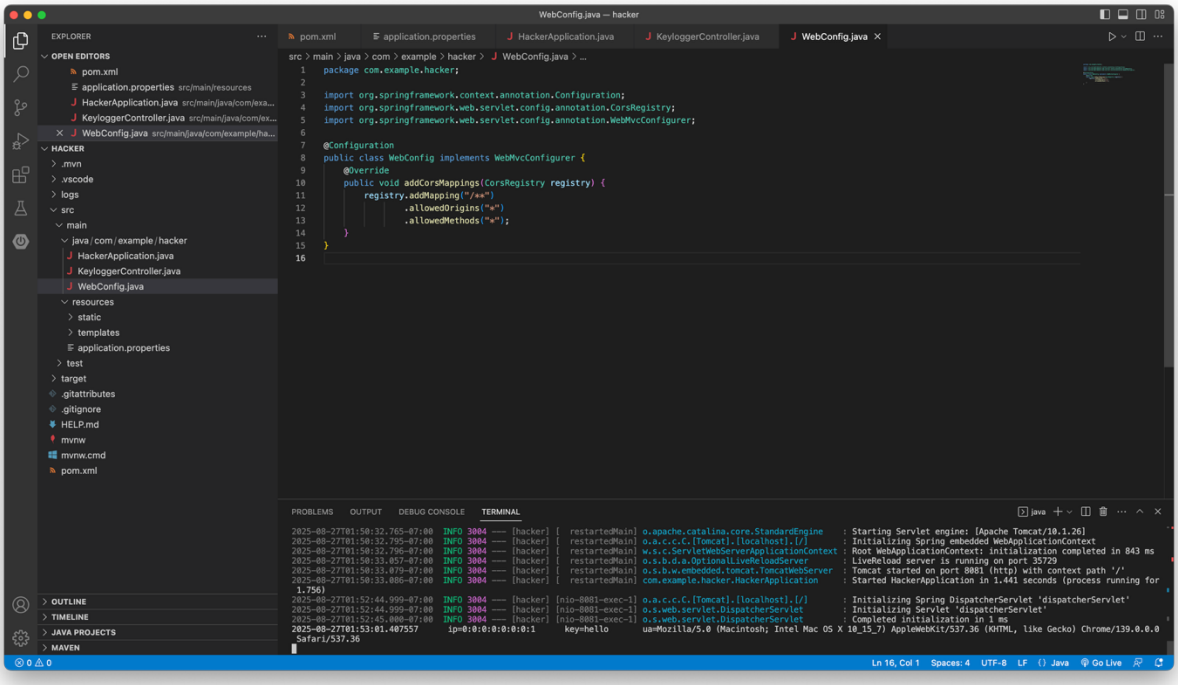

Figure 7 Console shows timestamp, client IP, and captured key value.

Keystrokes File Evidence

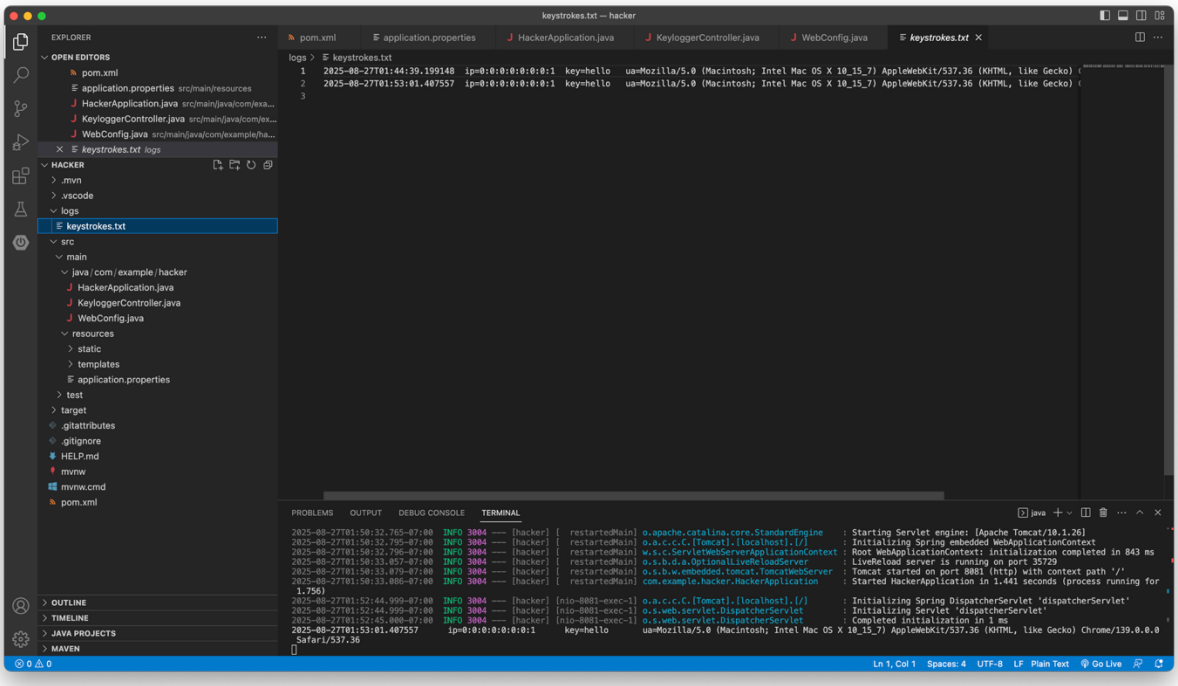

Figure 8 `logs/keystrokes.txt` contains appended entry for the test key.

Step 3 — Verify the XSS sink in the Comments app & prep the payload

3.1 Open the vulnerable view

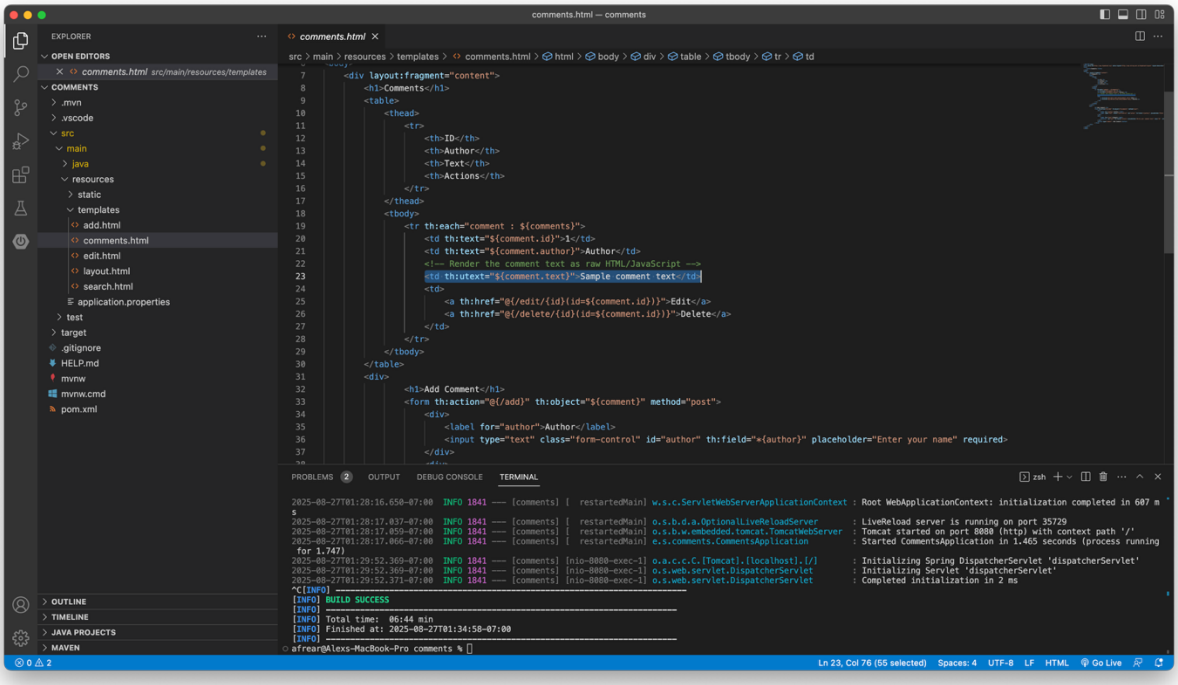

Figure 9 `comments.html` uses `th:utext` to render user input unescaped, enabling stored XSS.

3.2 Confirm the route that loads the page

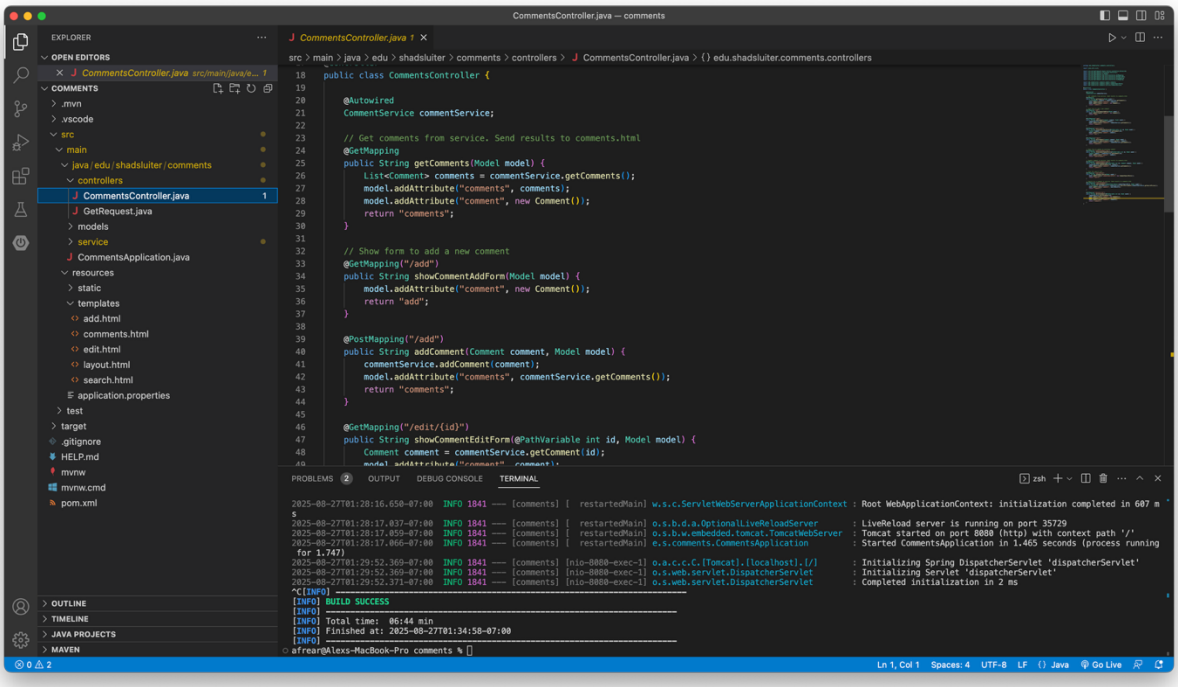

Figure 10 Controller method returns the `comments` template that displays unescaped comment text.

Step 4 — Inject the Stored XSS Keylogger Payload

4.1 Open the Add Comment form

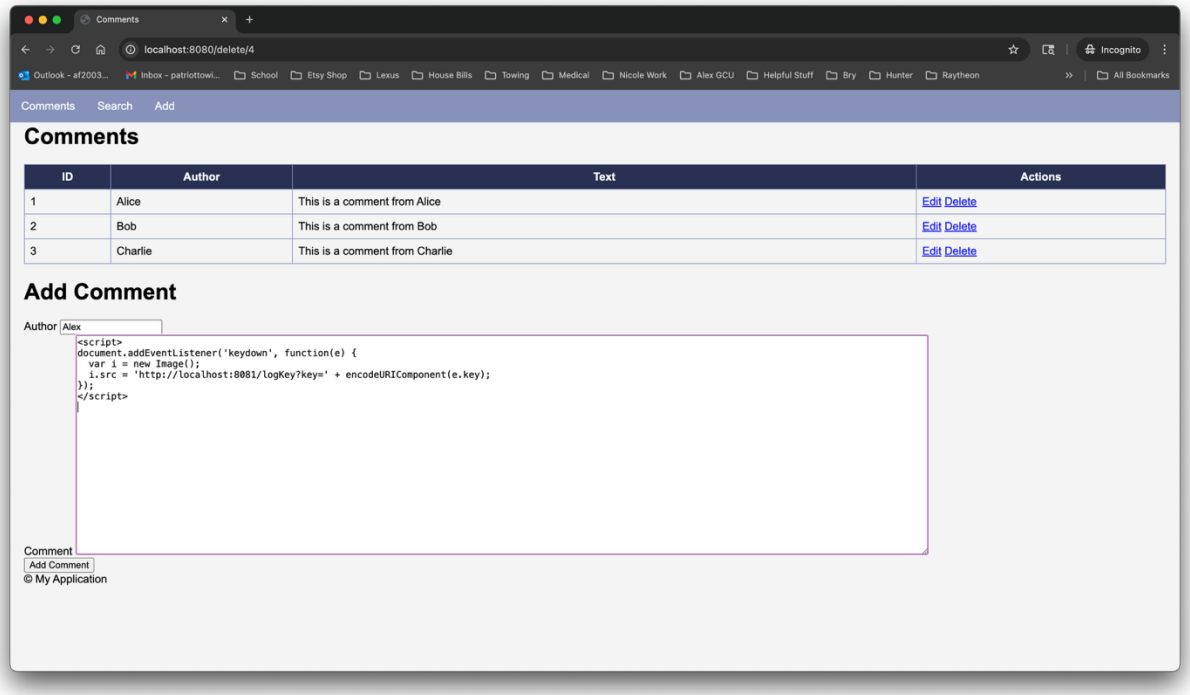

Figure 11 Add Comment form filled with the malicious keylogger payload in the Comment field.

4.2 Submit the payload

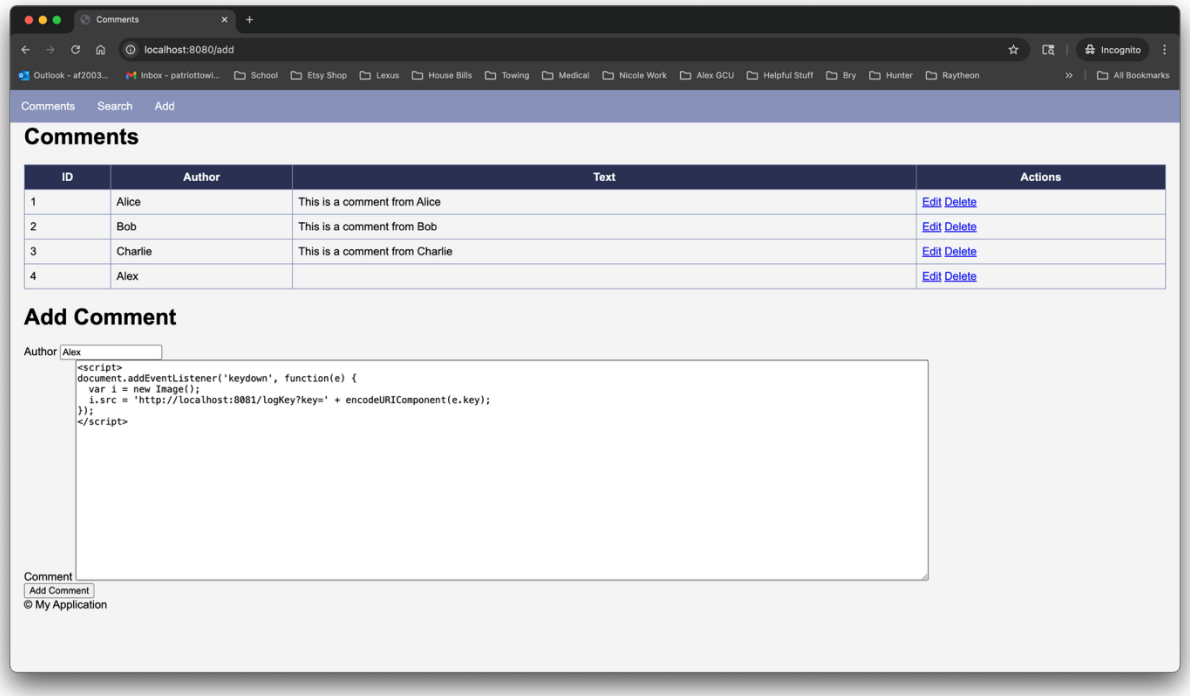

Figure 12 Comments page now contains the injected `<script>` payload displayed in the table.

4.3 Trigger some keystrokes

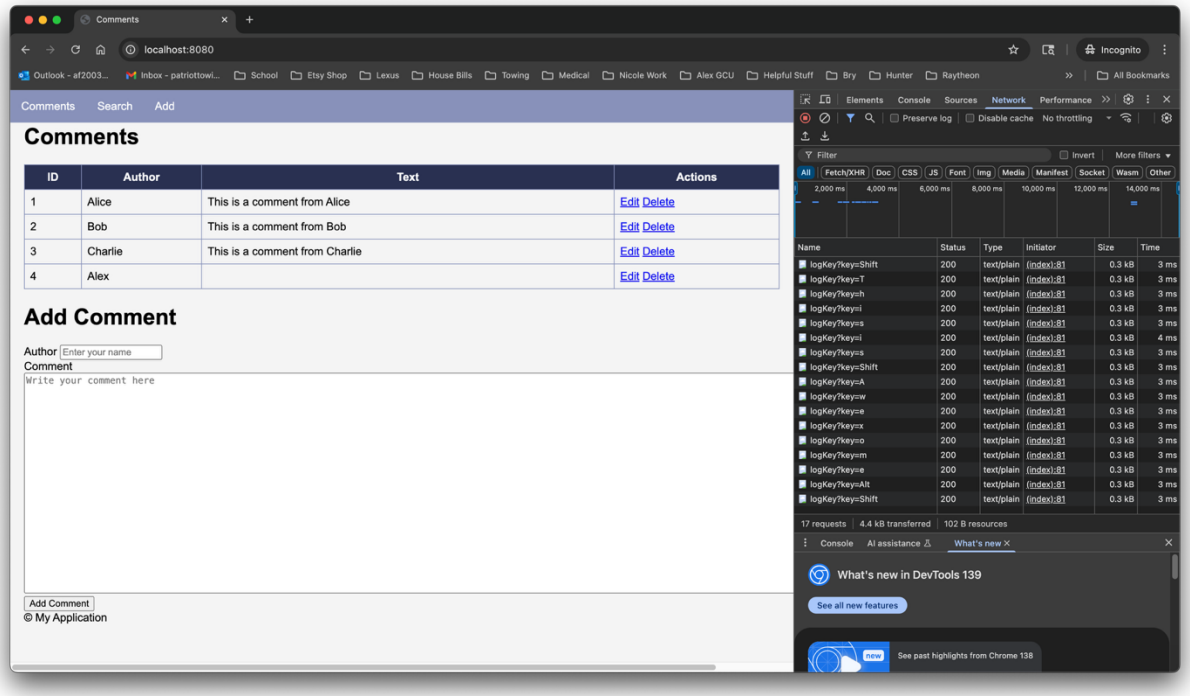

Figure 13 DevTools Network tab shows multiple requests to `/logKey?key=...` for each keystroke.

4.4 Verify in the listener console

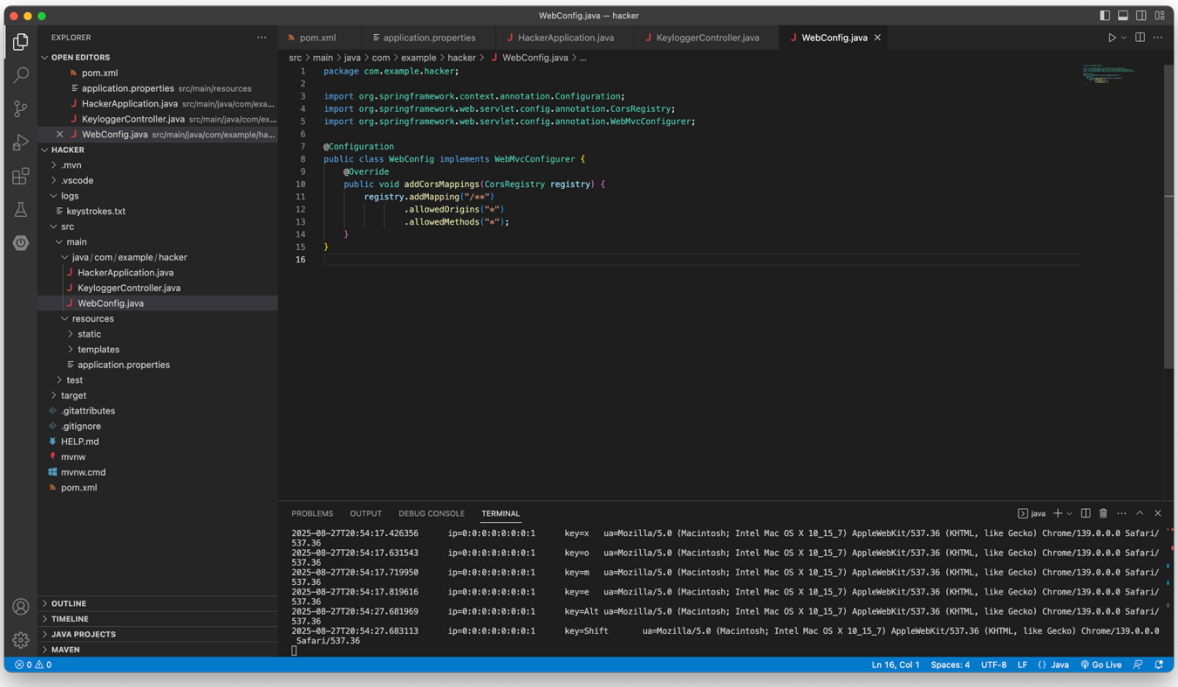

Figure 14 Console output shows each captured keystroke with timestamp, IP, and user agent.

4.5 Verify in the keystrokes log file

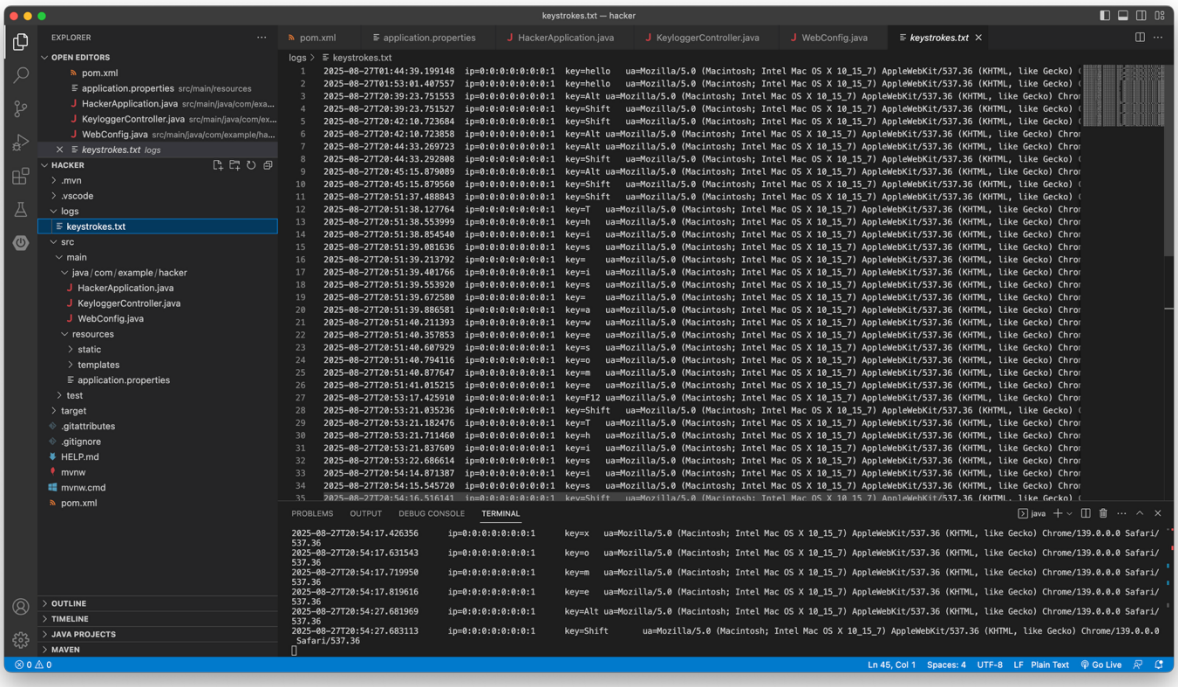

Figure 15 `logs/keystrokes.txt` contains entries for all keys typed on the Comments page.

Step 5 — Apply the Fix and Verify

5.1 Open the vulnerable template and apply the fix

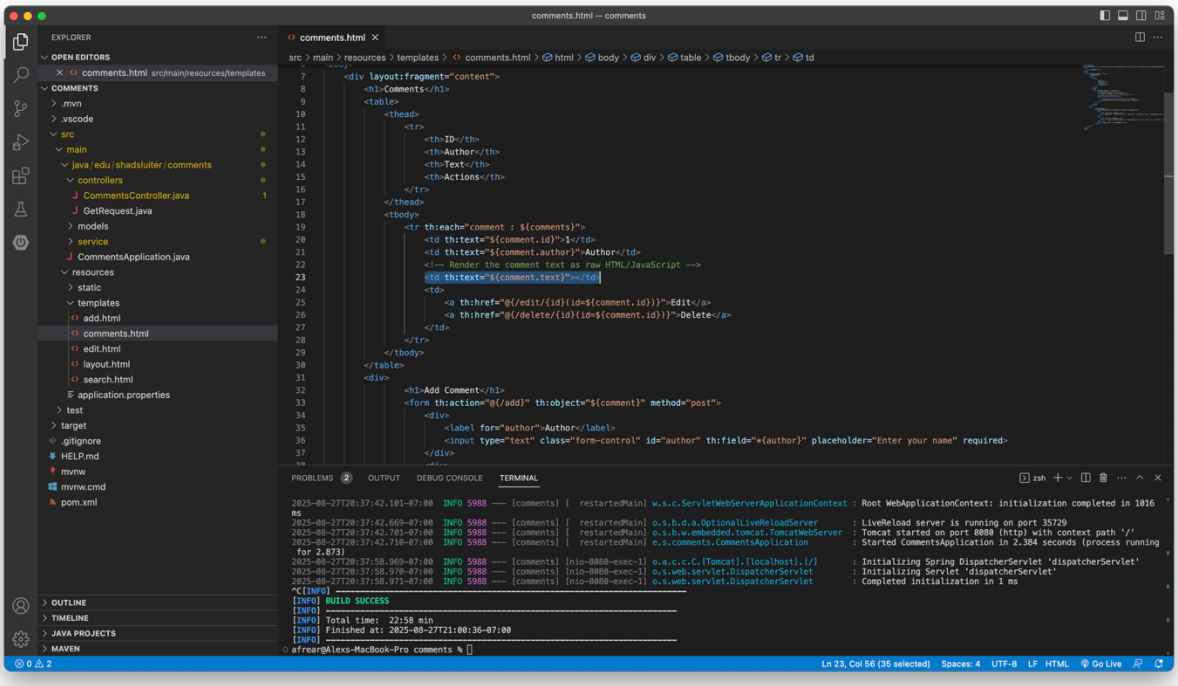

Figure 16 Updated template escapes user input, preventing script execution.

5.2 Restart the Comments app

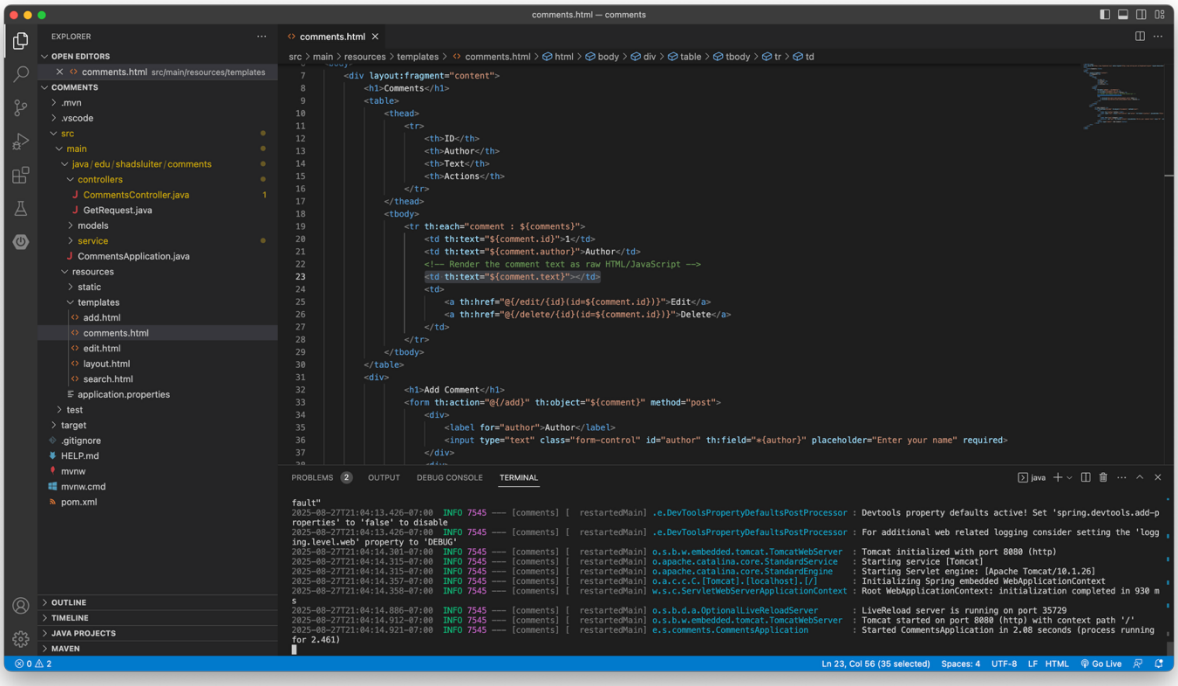

Figure 17 Console shows the Comments app restarted successfully after the fix.

5.3 Reload the Comments page

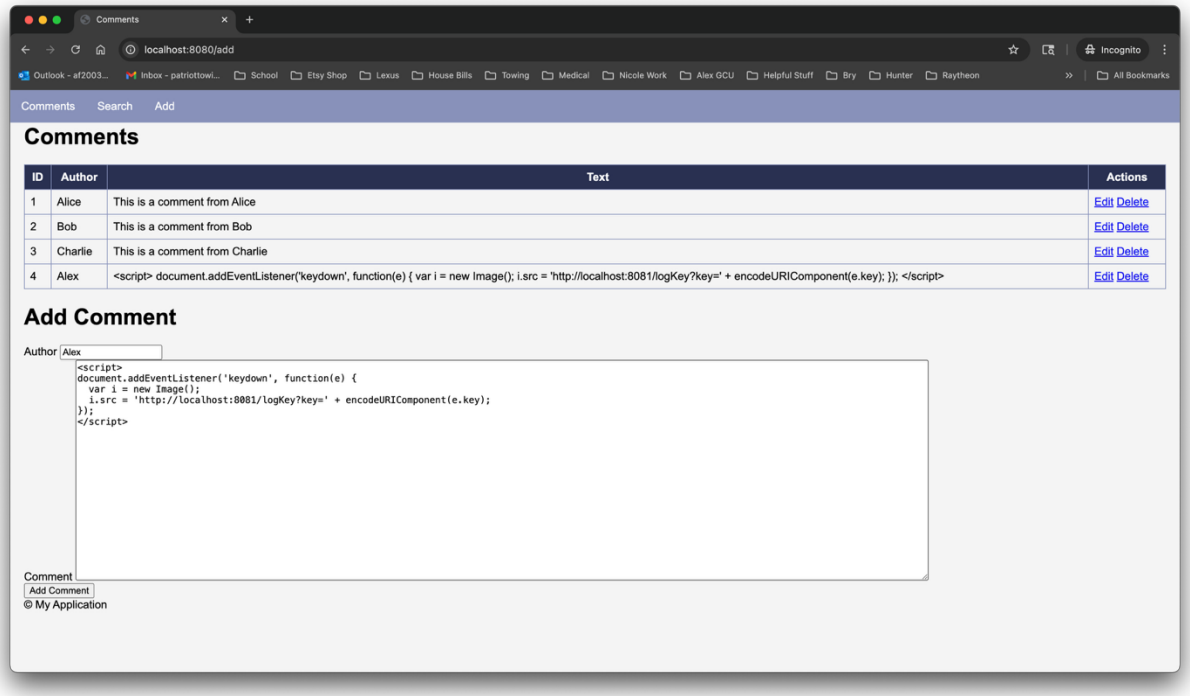

Figure 18 Comments page shows the script code as plain text instead of executing.

Step 6 — Add **OWASP Java Encoder** (sanitize on input)

6.1 Add the dependency to the **Comments** app

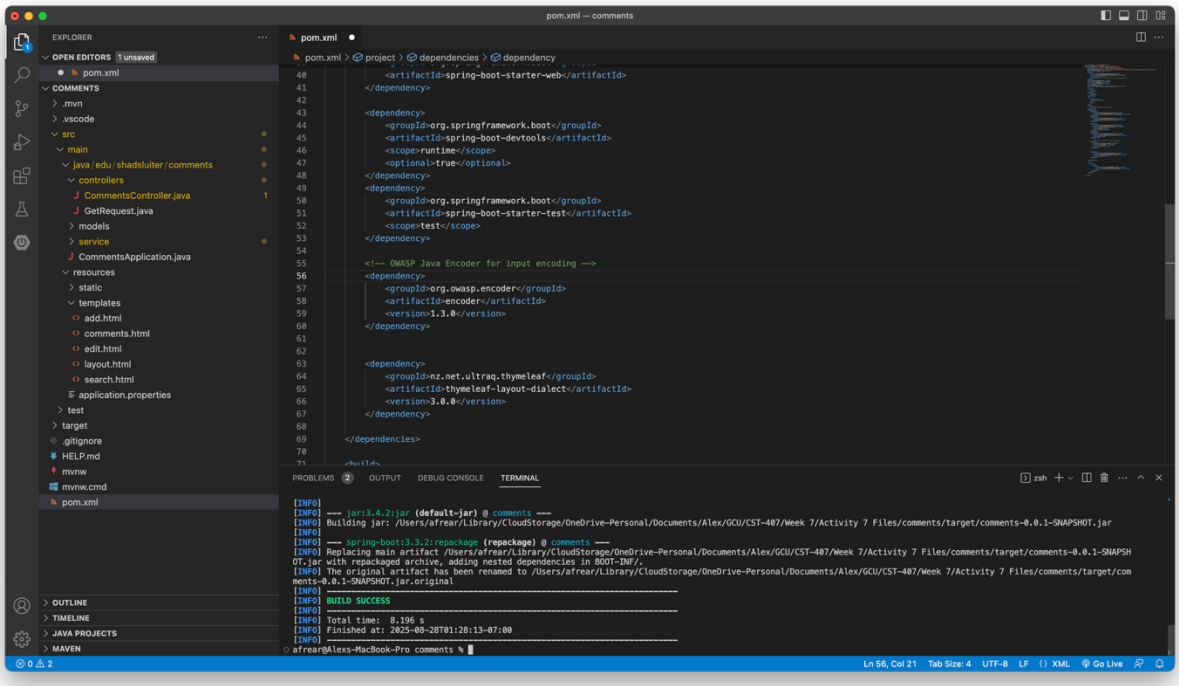

Figure 19 `pom.xml` includes `org.owasp.encoder:encoder` for input encoding.

6.2 Encode user input before saving (create & edit)

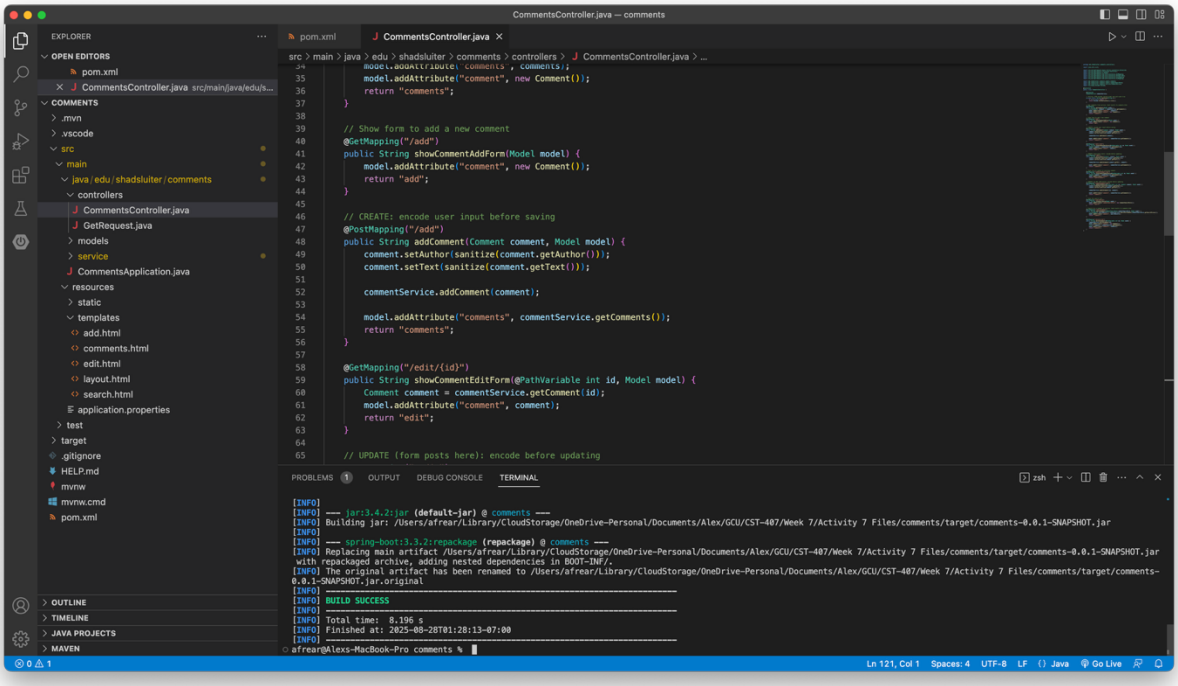

Figure 20 `Encode.forHtmlContent(...)` used to encode author and text before saving.

6.3 Rebuild & restart the **Comments** app

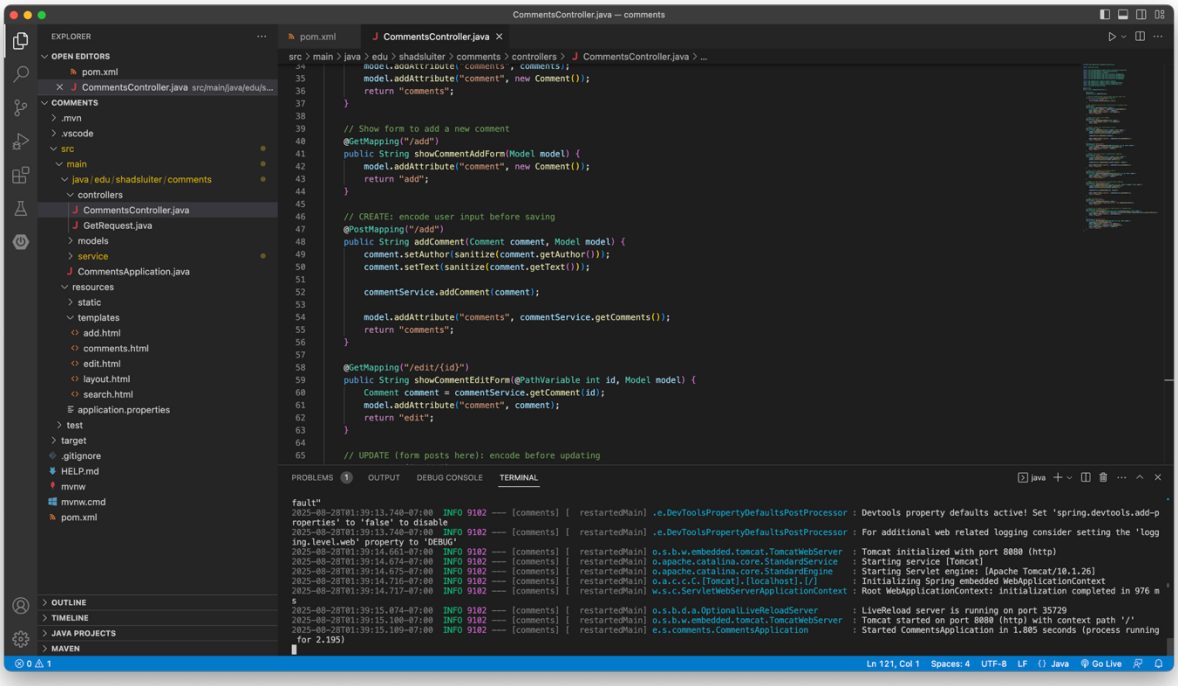

Figure 21 Application restarted after adding OWASP Encoder.

6.4 Try to inject a *new* script (post-fix test)

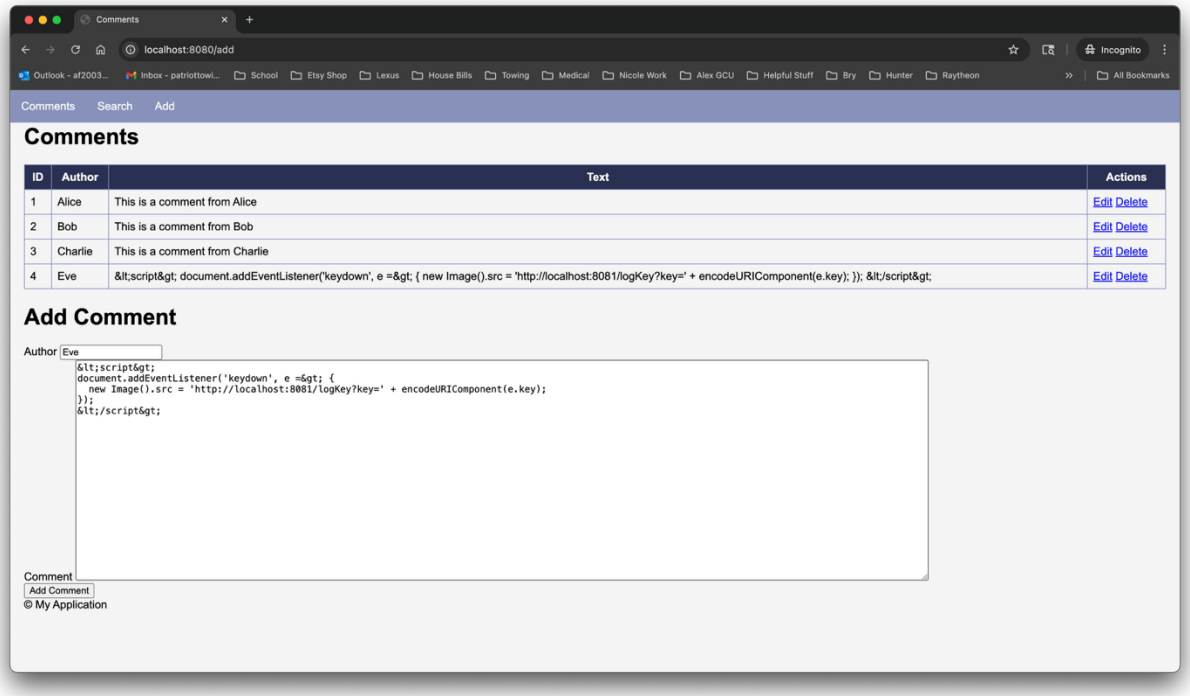

Figure 22 The newly added `<script>` comment is stored in encoded form and shown as plain text (no execution).

Summary of Key Concepts

In this activity, I demonstrated how a stored cross-site scripting (XSS) attack works by injecting a malicious `<script>` payload into the Comments application. Because the template originally rendered user input as raw HTML/JavaScript, the injected code executed in every visitor's browser, sending keystroke data to a separate "hacker" listener service on port 8081. I confirmed the attack by reviewing browser DevTools network traffic, the listener console logs, and the `keystrokes.txt` file.

To fix the vulnerability, I implemented two layers of defense. First, I updated the template to use `th:text` instead of `th:utext` so that user content would be displayed as escaped text by default. Second, I added the **OWASP Java Encoder** library to the project and updated the `CommentsController` to sanitize user inputs with `Encode.forHtml()` before storing them. With these changes, malicious scripts were neutralized and displayed safely as plain text, proving the application was no longer vulnerable.

Reflection

This lab reinforced how dangerous stored XSS can be, since the injected script executed automatically for every visitor who loaded the Comments page. It also showed how attackers can easily capture sensitive data like keystrokes with a separate listener service. By setting up this attack end-to-end, I saw how real the risk is when user input is not handled securely.

Applying the fixes emphasized the importance of **defense in depth**. Using `th:text` ensures safe output encoding at the view layer, while OWASP Encoder enforces sanitization before data is stored, reducing the risk of script injection from multiple angles. This dual approach highlights why developers should never trust user input and should always validate, sanitize, and escape data. The activity ultimately deepened my understanding of both exploiting and mitigating XSS vulnerabilities in web applications.